End-to-End CI/CD Pipeline for a Calculator REST API

CSE 816 Software Production Engineering — Mini-Project Report

Ramya Parsania (IMT2023015)

October 29, 2026

GitHub repository `https://github.com/RAMYA-PARSANIA/spe_mini_project`
Docker Hub image `https://hub.docker.com/r/itsrp/calculator`

1 Overview

A Spring Boot calculator REST API (Java 21, Maven) is wrapped in a fully automated pipeline: a push to GitHub triggers Jenkins, which builds, tests and packages the application, builds a Docker image tagged with the build number, pushes it to Docker Hub and deploys it with Docker Compose behind an Nginx reverse proxy. The build result is e-mailed. No step is manual.

Language / framework	Java 21, Spring Boot 3.5	Build & tests	Maven, JUnit 5, MockMvc
Version control	Git, GitHub	CI/CD	Jenkins (declarative `Jenkinsfile`)
Containers	Docker (multi-stage)	Deployment	Docker Compose, Nginx

2 Pipeline Architecture

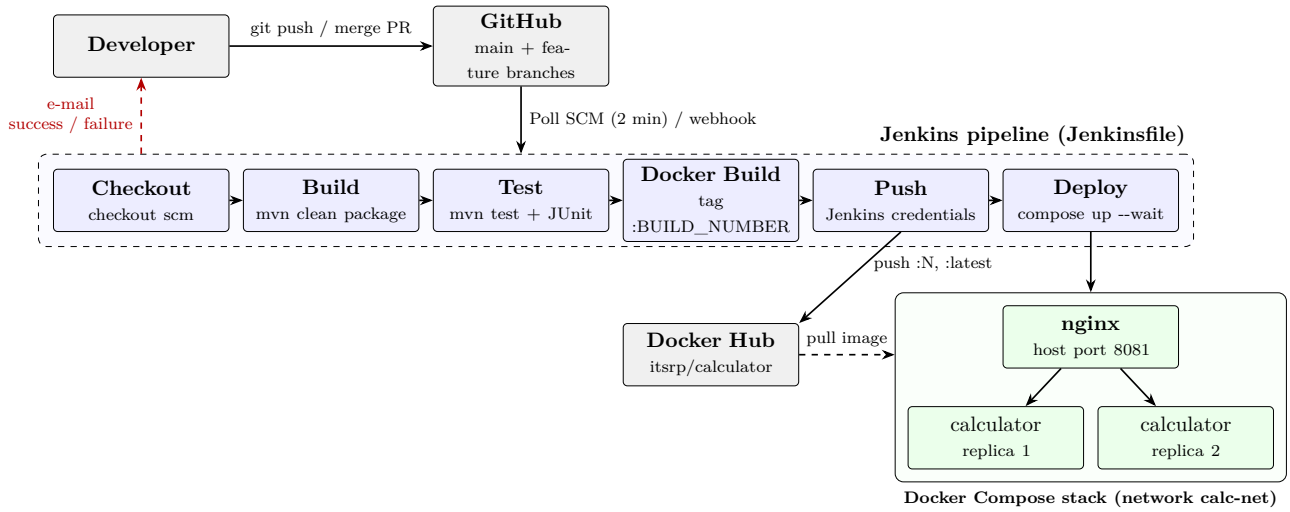


Figure 1: CI/CD flow. Jenkins runs on a Linux VM that also hosts the Docker engine and the deployed stack.

3 Application and Tests

Endpoints `GET /api/v1/{add,subtract,multiply,divide}?a=&b=` return JSON, e.g.

`GET /api/v1/add?a=2&b=3` → `{"operation":"add","a":2,"b":3,"result":5}`

and `/actuator/health` is the Spring Actuator probe used by the container health checks.

- **Layers:** `CalculatorController` (HTTP, input checks) → `CalculatorService` (arithmetic) ; a `@RestControllerAdvice` maps every error to HTTP 400 with a body `{timestamp, status, error, message, path}`.
- **Validation:** missing/empty parameters, non-numeric values, `NaN`/`Infinity` and division by zero all return 400 with a clear message.
- **Exact arithmetic:** `BigDecimal` with 16 significant digits, so $0.1 + 0.2$ returns `0.3` (with `double` it would be `0.30000000000000004`).
- **Web page:** a plain static `index.html` served at `/` with fields for `a` and `b` and an operation drop-down; it calls the API and shows the JSON response.
- **Tests (35, all via `mvn test`):** `CalculatorServiceTest` — parameterised JUnit 5 unit tests for every operation incl. divide-by-zero (21); `CalculatorControllerTest` — `@WebMvcTest` + MockMvc for JSON output and every 400 case (10); `CalculatorApplicationTests` — `@SpringBootTest` integration test hitting real endpoints, the web page and `/actuator/health` (4). Surefire writes JUnit XML to `target/surefire-reports`, which Jenkins publishes.

4 Pipeline Stages

Stage	What it does
Trigger	`pollSCM('H/2 * * * *')` checks GitHub every 2 minutes; `githubPush()` is also declared so a webhook works when Jenkins is publicly reachable.
Checkout	`checkout scm` fetches the commit that triggered the build.
Build	`mvn -B clean package -DskipTests`; the JAR is archived and fingerprinted.
Test	`mvn -B test`. `junit` publishes the XML reports (`post always`); a failing test makes Maven exit non-zero, which fails the build.
Docker Build	`docker build -t itsrp/calculator:$BUILD_NUMBER -t itsrp/calculator:latest .`
Push	`withCredentials` injects the Docker Hub access token stored in the Jenkins Credentials store (ID `dockerhub-credentials`); login uses `--password-stdin`, so the token never appears in the log. Both tags are pushed, then `docker logout`.
Deploy	`docker-compose up -d --remove-orphans --wait` with `IMAGE_TAG=$BUILD_NUMBER`; `--wait` blocks until every container is healthy. The pipeline then calls `/actuator/health` and `/api/v1/add` *through nginx*. On failure it redeploys the last successful build's image (rollback).
Post	`emailext` sends a success or failure mail (log attached on failure); dangling images are pruned.

Re-runnable and idempotent. The Compose project name is fixed (`name: calculator`), so every run updates the same stack; Compose only recreates containers whose image changed; `disableConcurrentBuilds()` stops two deploys racing; and a failed deploy rolls back. Running the job twice with no code change leaves the API serving throughout.

Version control. Work was committed progressively on `main` (ignore rules, API, tests) and on two feature branches, `feature/containerization` and `feature/jenkins-pipeline`, merged through pull requests #2 and #1. `.gitignore` excludes `target/`, IDE files, `.env` and key files; no credentials are in the repository.

5 Containerisation and Deployment

Dockerfile (multi-stage). Stage 1 (`maven:3.9.16-eclipse-temurin-21`) copies `pom.xml` first and runs `dependency:go-offline`, so the dependency layer is cached until the POM changes; it then copies `src/` and packages the JAR. Stage 2 (`eclipse-temurin:21-jre-alpine`) contains only the JRE and the JAR, runs as a non-root `app` user, declares `EXPOSE 8080` and a `HEALTHCHECK`, and sizes the heap

with `-XX:MaxRAMPercentage=75`. `.dockerignore` keeps `target/`, `.git/`, IDE files and secrets out of the build context. The image runs with a single `docker run -p 8080:8080 itsrp/calculator`.

Docker Compose. `docker-compose.yml` defines two services on the bridge network `calc-net`:

- `calculator` — 2 replicas, health check on `/actuator/health`, `restart: unless-stopped`, 512 MB memory limit, environment variables (port, JVM options, time zone). It is not published on the host; it is reachable only through the proxy.
- `nginx` — reverse proxy on host port 8081 with the configuration baked into a small image; it starts only after the API is healthy (`depends_on: service_healthy`) and spreads requests across both replicas (the `X-Upstream-Server` header shows which one answered).

6 Challenges Faced

Problem	Cause and fix
Jenkins on Windows could not use Docker	Jenkins ran as a Windows service (LocalSystem) while Docker Desktop's engine belongs to the logged-in user; switching the service to the user account failed because the account is a passwordless Microsoft account. *Fix:* moved Jenkins to a Linux VM alongside Docker Engine.
Build could not compile for Java 21	The VM had the Java 21 runtime but the Java 17 compiler (`javac`). *Fix:* installed `openjdk-21-jdk` and selected it with `update-alternatives`.
`permission denied` on the Docker socket	The `jenkins` user was not allowed to use Docker. *Fix:* added it to the `docker` group and restarted Jenkins.
`docker compose` not found	The VM has Compose v5.6.0 installed as the standalone `docker-compose` binary, not as the Docker CLI plugin. *Fix:* the Deploy stage calls `docker-compose`.
GitHub webhook cannot reach Jenkins	Jenkins runs on a VM inside a laptop with no public URL. *Fix:* Poll SCM every 2 minutes (a webhook via ngrok is possible since `githubPush()` is declared).
Proxy breaking after a redeploy	Nginx resolves upstream host names once at start-up, so recreated containers (new IPs) would give 502 errors. *Fix:* Docker's DNS resolver (`127.0.0.11`, 10 s validity) with a variable `proxy_pass`, so names are re-resolved; verified by redeploying with a new tag.